Setup Guide: BYO SSF Provider

The BYO SSF Provider is a web application that lets you send CAEP (Continuous Access Evaluation Protocol) Risk Change events to Okta via the Shared Signals Framework (SSF). A typical use case is testing how Okta's Identity Threat Protection responds to an external risk signal — for example, simulating a device risk change from a third-party security tool and verifying that Okta re-evaluates an active session or elevates the user's risk level accordingly.

The app walks you through a guided 4-step setup entirely through a browser UI: generating RSA signing keys, configuring your Okta connection, registering this app as a Security Events Provider in your Okta tenant, and sending risk events.

There are 2 primary hosting options:

- **Local Hosting** — Run the app on `localhost:3000` and expose it publicly via ngrok. Best for short-lived testing sessions.
- **Web Hosting** — Deploy to Vercel (or a similar platform) for a stable HTTPS URL. Recommended for demos or longer-running setups.

Prerequisites

Before starting, ensure you have:

- **Node.js 18+** installed
- An **Okta tenant** with admin access
- An **Okta API token** — generate one in the Okta Admin Console under **Security** → **API** → **Tokens**
- **ngrok** (for local hosting) or a **Vercel account** (for web hosting)

Step 1: Clone and Run the App

Clone the BYO SSF Provider GitHub repository and install dependencies:

```
git clone https://github.com/<your-org>/byo-ssf-transmitter
cd byo-ssf-transmitter
npm install
npm run dev
```

Open `http://localhost:3000` in your browser. You should see the BYO SSF Provider UI with the 4-step guided setup.

Connecting to Okta — HTTPS Requirement

Okta must be able to reach this app's JWKS endpoint over **HTTPS**. Standard HTTP URLs will be rejected. Depending on where the app is hosted, handle this as follows:

Local Hosting — Use ngrok to generate a secure tunnel:

```
ngrok http 3000
```

Copy the generated `https://` URL (e.g. `https://abc123.ngrok-free.app`). You will enter this as the **Public App HTTPS URL** in Step 3.

Web Hosting — Deploy to Vercel for a stable HTTPS URL with zero configuration. See [Step 1b: Deploy to Vercel](#) below.

Step 1b: Deploy to Vercel (optional)

For a more stable setup suitable for demos or repeated testing, deploy the app to Vercel.

1. Push the repository to GitHub if you haven't already.
2. Run the Vercel CLI from the project root:

   ```
   npx vercel
   ```

   Or connect your GitHub repository to Vercel at `https://vercel.com` for automatic deployments.
3. Once deployed, Vercel will provide an HTTPS URL (e.g. `https://byo-ssf-transmitter.vercel.app`).
4. In the Vercel dashboard, navigate to your project → **Settings** → **Environment Variables** and add the following variables. You will generate the key values in Step 2 of the in-app setup, then copy them from `.env.local`.

KEY	VALUE
`SSF_KEY_ID`	Key ID generated in Step 2
`SSF_PRIVATE_KEY`	RSA private key (PEM, newlines escaped as `\n`)
`SSF_PUBLIC_KEY`	RSA public key (PEM, newlines escaped as `\n`)
`APP_URL`	Your Vercel deployment URL (e.g. `https://byo-ssf-transmitter.vercel.app`)

5. Redeploy after adding the environment variables so they take effect.

Step 2: Generate Signing Keys

Every Security Event Token (SET) sent to Okta must be cryptographically signed so Okta can verify it came from this app. The app uses an RSA 2048-bit key pair for this purpose.

1. In the BYO SSF Provider UI, locate **Step 1 — Generate Signing Keys**.
2. Click the **Generate Signing Keys** button.

   The app generates a new RSA key pair and writes it directly to `.env.local` in the project directory. No terminal commands are required.
3. Once complete, the panel will show a **Configured** badge and display the generated **Key ID**.

   > ***Note:*** *If you are deploying to Vercel, copy the values from `.env.local` into your Vercel environment variables (see Step 1b) and redeploy before continuing.*

4. The public key is automatically published at `/api/jwks`. Okta fetches this endpoint to verify the signature on every event token you send.

Endpoint	Description
`<app-url>/api/jwks`	JWKS public key endpoint (used by Okta to verify tokens)
`<app-url>/.well-known/ssf-configuration`	SSF discovery document

Step 3: Configure Okta Connection

This step connects the app to your Okta tenant so it can authenticate API calls and register itself as a Security Events Provider.

1. In the BYO SSF Provider UI, locate **Step 2 — Configure Okta Connection**.
2. Click on the **Configuration** panel header to expand it.
3. Fill in the following fields:

Field	Value
Okta Tenant URL	Your Okta organization's base URL, e.g. `https://your-org.okta.com`. Found in your Okta Admin Console address bar.
Okta API Key	An Okta API token (SSWS token). Generate one under **Security** → **API** → **Tokens** in the Okta Admin Console.
Public App HTTPS URL	The publicly reachable HTTPS URL for this app — either your ngrok tunnel URL or your Vercel deployment URL.

> ***Note:*** *The Public App HTTPS URL must be HTTPS. Okta fetches the JWKS endpoint at* `{url}/api/jwks` *to verify JWT signatures. The API Key is stored in your browser's local storage only and is never transmitted to any server other than your own Okta tenant.*

4. Click **Save Configuration**. The panel will collapse and show a **Configured** badge when all three fields are valid.

Step 4: Create the Security Events Provider in Okta

This step registers the BYO SSF Provider app with your Okta tenant via the API, creating a stream that Okta will use to receive security signals from this app.

1. In the BYO SSF Provider UI, locate **Step 3 — Create Security Events Provider**.
2. The **Provider Name** field defaults to `BYO SSF Provider`. You may change this to any name that will help identify it in the Okta Admin Console.
3. The panel displays the **Well-known URL** that will be registered with Okta:

```
<app-url>/.well-known/ssf-configuration
```

4. Click **Create Provider in Okta**.

The app calls the Okta API at `/api/v1/security-events-providers`, first attempting to register using the SSF well-known URL, then falling back to issuer + JWKS URL if needed. If a provider with this issuer already exists in Okta, the existing registration is surfaced rather than returning an error.

5. On success, the panel shows a **Created** badge and displays the Provider ID returned by Okta.

> ***Verify in Okta:*** *Navigate to the Okta Admin Console →* ***Security*** *→* ***Device Integrations*** *→* ***Receive Shared Signals****. You should see the provider listed with a status of* ***Active****.*

The screenshots below show what to expect in the Okta Admin Console after successful registration:

- The **Receive Shared Signals** tab listing the new stream with status Active.
- The stream configuration dialog confirming the Well-known URL.

Step 5: Send a Risk Event

With the provider registered, you can now send CAEP Risk Change events to Okta.

1. In the BYO SSF Provider UI, locate **Step 4 — Send Risk Events**.

2. Fill in the **Send Risk Event** form:

Field	Description
User Identifier	The email address or Okta User ID of the user whose risk level is changing.
Identifier Type	Select **Email** or **Okta User ID** from the dropdown.
Previous Level	The user's current risk level before this event. Choose from: `low`, `medium`, `high`, `critical`.
Current Level	The new risk level to signal to Okta. Must differ from Previous Level.
Reason	Optional free-text description of why the risk level changed.

3. Click **Send Risk Event**.

   The app signs a Security Event Token (SET) using the RSA private key from Step 2, then POSTs it to Okta's SSF endpoint:

```
POST <okta-tenant>/security/api/v1/security-events
Content-Type: application/secevent+jwt
```

4. On success, the **Last Sent SET Token** panel expands below the form, showing:

   - **Decoded Payload (JSON)** — the full JWT claims, including the event type, subject, risk levels, and timestamps.
   - **Signed JWT** — the raw compact JWT that was transmitted to Okta.

   An example decoded payload looks like this:

```
{
  "iss": "https://your-app.example.com",
```

```
  "aud": "https://your-okta-org.okta.com",
  "iat": 1706000000,
  "jti": "evt_abc123",
  "events": {
    "https://schemas.openid.net/secevent/caep/event-type/assurance-level-change": {
      "subject": {
        "format": "email",
        "email": "user@example.com"
      },
      "current_level": "high",
      "previous_level": "low",
      "change_direction": "increase",
      "event_timestamp": 1706000000
    }
  }
}
```

5. Each sent event is added to the **Event History** panel at the bottom of the page, showing the timestamp, user, risk level transition, and the Okta HTTP response status.

Step 6: Verify in the Okta System Log

After sending an event, confirm that Okta received and processed it.

1. In the Okta Admin Console, navigate to **Reports** → **System Log**.
2. Search or filter for the event type:

   ```
   security.events.provider.receive_event
   ```

3. You should see an entry for each event sent, with the subject user and the event details matching what you transmitted from the BYO SSF Provider app.

Troubleshooting

Symptom	Likely Cause	Resolution
Provider creation fails with "Invalid URL"	The Public App HTTPS URL is HTTP or unreachable	Ensure ngrok or Vercel is running and the URL is HTTPS
Provider creation fails with "already exists"	A provider with this issuer is already registered in Okta	The app will surface the existing provider automatically — no action needed
Event send fails with 401	The provider's JWKS endpoint returned the wrong key	Regenerate keys in Step 2, re-create the provider in Step 4
Event send fails with 400	Invalid user identifier or matching risk levels	Ensure the identifier is a valid email or Okta User ID, and that Previous ≠ Current level
No entry in Okta System Log	Event was not received by Okta	Check that the provider status is Active in Device Integrations

App Endpoints Reference

Endpoint	Method	Description
`/.well-known/ssf-configuration`	GET	SSF discovery document served to Okta during provider registration
`/api/jwks`	GET	JWKS public key endpoint — Okta fetches this to verify SET signatures
`/api/generate-keys`	POST	Generate a new RSA key pair and write to `.env.local`
`/api/create-provider`	POST	Register this app as a Security Events Provider in Okta
`/api/transmit`	POST	Sign and send a SET to Okta